

|                                                                                                                                                                                                                                    |                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p><b>ESPE</b><br/>UNIVERSIDAD DE LAS FUERZAS ARMADAS<br/>INNOVACIÓN PARA LA EXCELENCIA</p> <p><b>TEMA:</b></p> <p><b>Patrones de Diseño</b></p> | Departamento: Departamento de Ciencias de la Computación                                                                                           |
|                                                                                                                                                                                                                                    | Carrera: Ingeniería de Software<br>Asignatura: Análisis y Diseño de Software<br>NRC: 30723                                                         |
|                                                                                                                                                                                                                                    | Integrantes Grupo 3: <ul style="list-style-type: none"> <li>• Ayuquina Danny.</li> <li>• Cañarte Saray.</li> <li>• Villagómez Doménica.</li> </ul> |

## PROYECTO: FICITAS

### 1. Patrones de Diseño

#### 1.1. Decorator

La elección del patrón Decorator para FiCitas responde a un principio fundamental de la ingeniería de software: el principio de responsabilidad única (SRP). Los servicios centrales del sistema —autenticación, citas y pacientes— se concentran exclusivamente en las operaciones de negocio y persistencia de datos. Incorporar validaciones y verificaciones de disponibilidad directamente en estas clases violaría este principio, generando componentes sobrecargados y difícilmente testeables de forma aislada.

El patrón Decorator resuelve este problema mediante composición en lugar de herencia. Cada decorador mantiene una referencia al servicio que envuelve (`this.servicioBase`) y reproduce la misma firma de métodos que este expone, delegando la llamada una vez superadas las validaciones correspondientes. Esto garantiza que el cliente sea transparente frente a la composición: las vistas que construyen la cadena —VistaLogin, VistaRecuperarContraseña, VistaCambiarContraseña, VistaPacientes y VistaCitas— interactúan únicamente con el decorador resultante (`new ValidacionDecorator(new ServicioAutenticacion(...))`, `new DisponibilidadDecorator(new ServicioCitas(...))`), sin conocer si por debajo existe una o varias capas de lógica. Esta transparencia hace que el sistema sea abierto a la extensión y cerrado a la modificación (principio OCP), ya que agregar una nueva capa —por ejemplo, un decorador de autorización de roles— no requeriría modificar ninguna clase existente, solo envolver el servicio base con la nueva capa.

La aplicación del patrón es específica según el dominio: `ValidacionDecorator` envuelve a `ServicioAutenticacion` y a `ServicioPacientes`, encapsulando reglas que cambian con frecuencia (formato de correo, detección de contenido no permitido, validación de cédula ecuatoriana, formato de teléfono y consistencia de fecha de nacimiento) antes de delegar la operación. `DisponibilidadDecorator`, por su parte, envuelve exclusivamente a `ServicioCitas`, verificando la duración de la cita, que la fecha y hora no sean pasadas, que el paciente exista y esté activo, y que no exista solapamiento de horario en el consultorio solicitado. Los servicios `ServicioPerfiles`, `ServicioUsuarios` y `ServicioPlanificacion` ejecutan sus propias reglas de validación internamente y no forman parte de esta cadena de decoradores, ya que sus reglas de negocio están estrechamente ligadas a su propia lógica de persistencia y no se benefician de una capa de validación desacoplada.

Finalmente, la separación de responsabilidades facilita las pruebas unitarias: cada decorador puede probarse inyectándole un mock del servicio subyacente (servicioBase), validando de forma aislada tanto las reglas de validación y disponibilidad como la lógica de negocio de los servicios envueltos, sin que unas interfieran con las otras. Esto reduce el acoplamiento entre capas y mejora la mantenibilidad del sistema a largo plazo.

| Elemento del patrón  | Clase en el proyecto                                                                     | Responsabilidad                                                                                                                                                  |
|----------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Concrete Component   | ServicioAutenticacion, ServicioPacientes, ServicioCitas                                  | Lógica de negocio y persistencia, sin validación de entrada                                                                                                      |
| Concrete Decorator 1 | ValidacionDecorador                                                                      | Valida formato, inyección de script y reglas de negocio antes de delegar. Envuelve ServicioAutenticacion (login/recuperación) y ServicioPacientes (alta/edición) |
| Concrete Decorator 2 | DisponibilidadDecorador                                                                  | Verifica duración, fecha futura, paciente activo y solapamiento de horario/consultorio. Envuelve únicamente ServicioCitas                                        |
| Client               | VistaLogin, VistaRecuperarContrasena, VistaCambiarContrasena, VistaPacientes, VistaCitas | Construyen la cadena (new Decorador(new Servicio(...))) e interactúan solo con el decorador                                                                      |

ServicioPerfiles, ServicioUsuarios y ServicioPlanificacion no usan Decorator: cada uno valida sus propios campos internamente (p. ej. ServicioPerfiles valida nombre/descripción/permisos duplicados sin una capa externa).

## 2.2.Observer

El patrón Observer es la solución más adecuada cuando un sistema necesita propagar efectos secundarios ante eventos sin que los emisores de esos eventos conozcan ni dependan de quienes los procesan. En FiCitas, los servicios de negocio (ServicioAutenticacion, ServicioCitas, ServicioPerfiles, ServicioUsuarios y ServicioPlanificacion) son responsables de ejecutar las operaciones del dominio, pero no es su responsabilidad saber que existe un sistema de auditoría ni un sistema de notificaciones. Codificar esas dependencias directamente en cada servicio los convertiría en componentes altamente acoplados, difíciles de mantener y de extender.

Al introducir GestorEventos como sujeto observable central, se invierte esta dependencia: cada servicio solo necesita publicar un evento (gestor.notificar({ tipo, datos })) tras cada operación relevante —inicio de sesión, registro o cancelación de una cita, alta o modificación de un perfil o usuario, y las distintas operaciones de planificación—, y el gestor se encarga de notificar a todos los observadores registrados. Esto respeta el principio de inversión de dependencias (DIP) y permite que el sistema de auditoría y el de notificaciones evolucionen de forma completamente independiente. El evento en sí se representa como un objeto simple con dos campos, tipo y datos; la mayor parte de los servicios publica

directamente un objeto literal con esa forma, y GestorEventos notifica a cualquier observador suscrito que exponga un método actualizar(evento), sin exigir formalmente que dicho observador herede de una clase o interfaz común.

La clase LogAuditoriaObserver justifica plenamente este enfoque: la trazabilidad completa de todas las acciones del sistema —quién hizo qué, cuándo y sobre qué datos— es un requisito transversal que afecta a todos los módulos. Esta clase implementa el contrato IObservadorAuditoria y, ante cualquier evento recibido, extrae el usuario, el perfil, la acción realizada y los datos anteriores/nuevos, persistiendo el registro en AuditoriaRepo. Implementarla mediante Observer garantiza que ningún servicio pueda olvidarse de registrar una acción, porque el registro ocurre de forma automática y centralizada ante cualquier evento publicado, lo que hace que el mecanismo de auditoría sea robusto frente a errores de omisión y coherente en todo el sistema.

Por su parte, NotificacionObserver demuestra otro beneficio clave del patrón: la capacidad de añadir nuevas reacciones ante eventos existentes sin modificar los emisores. Este observador filtra únicamente los eventos cuyo tipo comienza con CITA\_ y delega el procesamiento a ServicioNotificaciones, que selecciona el canal de envío (correo o WhatsApp) según la preferencia registrada del paciente. Si en el futuro se requiere una nueva reacción ante la creación de una cita —por ejemplo, un resumen semanal o una integración con un sistema externo—, bastará con registrar un nuevo observador en GestorEventos, sin modificar ServicioCitas ni ningún otro emisor. La combinación con el patrón Strategy en los canales de notificación (ICanal, CanalEmail, CanalWhatsApp) refuerza aún más esta extensibilidad, permitiendo que el canal de entrega sea configurable sin afectar la lógica del observador.

En conjunto, el uso del patrón Observer permite que FiCitas cumpla sus requisitos de auditoría y notificación de manera desacoplada, extensible y verificable, alineándose con los principios SOLID y con las buenas prácticas de arquitectura de software orientada a objetos.

| Elemento del patrón  | Clase en el proyecto                                       | Responsabilidad                                                                                                                                                             |
|----------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Subject / Observable | GestorEventos                                              | Mantiene la lista de observadores; notificar(evento) los invoca a todos                                                                                                     |
| Observer (interfaz)  | IObservadorAuditoria                                       | Contrato actualizar(evento): void                                                                                                                                           |
| Concrete Observer 1  | LogAuditoriaObserver (sí extiende la interfaz)             | Persiste todo evento en AuditoriaRepo: usuario, perfil, acción, datos anteriores/nuevos                                                                                     |
| Concrete Observer 2  | NotificacionObserver (duck-typed, no extiende la interfaz) | Filtra eventos CITA_* y delega el envío a ServicioNotificaciones                                                                                                            |
| Event / Value Object | Evento ({tipo, datos})                                     | Más simple que lo documentado originalmente; la mayoría de servicios publican objetos planos {tipo, datos} — solo ServicioPerfiles instancia new Evento(...) explícitamente |

|                                      |                                                                                                             |                                                                                           |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Publishers<br>(Concrete<br>Subjects) | ServicioAutenticacion,<br>ServicioCitas,<br>ServicioPerfiles,<br>ServicioUsuarios,<br>ServicioPlanificacion | Cada uno llama gestor.notificar(...) tras una<br>operación relevante                      |
| Strategy<br>(interfaz)               | ICanal                                                                                                      | Contrato enviar(mensaje): boolean                                                         |
| Concrete<br>Strategy 1/2             | CanalEmail /<br>CanalWhatsApp                                                                               | Implementan el envío por cada canal<br>(simulado con console.log, ambos retornan<br>true) |
| Contexto de<br>Strategy              | ServicioNotificaciones                                                                                      | Selecciona la estrategia según el<br>canalPreferido del paciente y ejecuta<br>enviar()    |
| Repos de<br>soporte                  | AuditoriaRepo (Mongo),<br>NotificacionRepo<br>(memoria)                                                     | Persisten auditoría y notificaciones<br>respectivamente                                   |